

Corporación Universitaria Iberoamericana

Programa de Desarrollo de Software

Nexus Talent

**Plataforma Web de Portafolios Digitales con Sistema de
Mentoría y Retroalimentación entre Pares**

Actividad 3 — Segunda Entrega del Proyecto de Software

Presentado por:

Nelson Andrés Ramírez Gutiérrez

Repositorio: <https://github.com/AndresRamirez0/nexus-talent>

Figma: <https://www.figma.com/design/oHiy8k9KpJe9BLXFkanOj3/Nexus-Talent>

Docente:

Tatiana Cabrera

Curso: Proyecto de Software

Bogotá, Colombia — 2026

SEGUNDA ENTREGA

Nexus Talent — Proyecto de Software

Sprint 1 — Prototipo funcional y arquitectura del sistema

Introducción

El presente documento corresponde a la segunda entrega del proyecto de software Nexus Talent, desarrollado en el marco de la asignatura Proyecto de Software del programa de Desarrollo de Software de la Corporación Universitaria Iberoamericana. Esta entrega da continuidad al proceso iniciado en la primera actividad, en la cual se realizó la identificación del problema, la formulación del proyecto, el diseño de la arquitectura conceptual y la definición de los requisitos del sistema.

En esta segunda fase se avanza desde el diseño conceptual hacia la construcción técnica del sistema. El objetivo central es transformar el prototipo de alta fidelidad validado previamente en una estructura técnica viable, con funcionalidades reales implementadas y operativas. Para ello se definió la arquitectura del sistema mediante el modelo C4, se tomaron y documentaron decisiones técnicas fundamentales a través de Architecture Decision Records (ADR), y se implementaron las dos primeras funcionalidades del producto: el módulo de autenticación completo con JWT y el módulo de gestión de perfil de usuario.

El desarrollo se realizó bajo la metodología ágil Scrum, completando el Sprint 1 en un período de dos semanas. El sistema implementado utiliza Node.js con Express en el backend, React en el frontend, PostgreSQL como base de datos relacional y JWT para la autenticación segura. La documentación de la API fue generada automáticamente mediante Swagger UI, y el código fuente está disponible públicamente en el repositorio de GitHub del proyecto.

El documento se organiza en seis secciones que cubren los entregables requeridos: actualización del MVP con historias de usuario, descripción del prototipo funcional con evidencias de pruebas

de usabilidad, arquitectura C4 en sus niveles 1 y 2, documento ADR con las decisiones técnicas adoptadas, descripción del repositorio del proyecto y retrospectiva del Sprint 1 completado.

Sección 1. Actualización del MVP e Historias de Usuario

1.1 Ajustes realizados al MVP

Con base en el proceso de validación inicial del prototipo conceptual desarrollado en la primera entrega, se realizaron los siguientes ajustes y mejoras al Producto Mínimo Viable de Nexus Talent durante el Sprint 1:

Mejoras en el módulo de autenticación: el formulario de registro fue mejorado significativamente incorporando validaciones en tiempo real que orientan al usuario durante el proceso de creación de cuenta. Se agregó un indicador visual de fortaleza de contraseña que muestra el nivel de seguridad en cuatro niveles: débil, regular, buena y fuerte. Se implementó la visualización opcional de la contraseña mediante un botón de ojo que permite al usuario verificar lo que está escribiendo. Se agregó un campo de confirmación de contraseña con validación en tiempo real que indica cuando ambas contraseñas coinciden. Los mensajes de error fueron rediseñados para ser específicos por campo, indicando exactamente qué requisito no se cumple en lugar de mostrar un mensaje genérico.

Requisitos de contraseña implementados: mínimo 8 caracteres, al menos una letra mayúscula, al menos una letra minúscula, al menos un número y al menos un carácter especial (@\$!%*?&).

Mejoras en la experiencia de usuario: la interfaz fue alineada con los colores corporativos de Nexus Talent (azul #2E5BA8), aplicando consistencia visual en todos los componentes. Se implementó

navegación protegida mediante rutas privadas que redirigen al usuario no autenticado al login automáticamente.

1.2 Historias de usuario priorizadas para el Sprint 1

Las siguientes seis historias de usuario fueron priorizadas e implementadas en el Sprint 1, con sus criterios de aceptación verificados en el prototipo funcional:

Tabla. *Resumen de historias de usuario implementadas en Sprint 1*

ID	Historia	Prioridad	Estado
HU01	Registro en la plataforma con validaciones en tiempo real	Alta	✓ Implementada
HU02	Inicio de sesión seguro con JWT	Alta	✓ Implementada
HU03	Gestión del perfil profesional	Alta	✓ Implementada
HU04	Visualización del perfil público	Alta	✓ Implementada
HU05	Navegación protegida por autenticación	Alta	✓ Implementada
HU06	Cierre de sesión seguro	Media	✓ Implementada

HU01 — Registro en la plataforma

Como estudiante o profesional joven quiero crear una cuenta en Nexus Talent proporcionando mis datos personales y rol para acceder a todas las funcionalidades de la plataforma y comenzar a construir mi portafolio digital profesional.

Criterios de aceptación: el sistema valida formato de correo y unicidad en base de datos; la contraseña cumple requisitos de seguridad validados en tiempo real; el indicador de fortaleza de contraseña es visible durante la escritura; el campo de confirmación valida la coincidencia; el

sistema genera token JWT al completar el registro; los mensajes de error son específicos por campo.

HU02 — Inicio de sesión seguro

Como usuario registrado quiero iniciar sesión con mi correo y contraseña para acceder a mi cuenta y funcionalidades personalizadas de la plataforma.

Criterios de aceptación: el sistema autentica usuario con email y contraseña; genera token JWT con expiración de 24 horas; redirige automáticamente al perfil tras autenticación exitosa; muestra mensaje de error claro ante credenciales inválidas; botón para mostrar u ocultar contraseña disponible.

HU03 — Gestión del perfil profesional

Como usuario registrado quiero editar mi perfil actualizando nombre, descripción profesional y habilidades para mantener mi información actualizada ante la comunidad.

Criterios de aceptación: el usuario puede editar nombre, descripción y habilidades; las habilidades se ingresan separadas por coma y se almacenan como array; los cambios se reflejan inmediatamente; el sistema confirma guardado exitoso; el campo nombre es obligatorio.

HU04 — Visualización del perfil público

Como reclutador o visitante quiero consultar el perfil de un usuario sin necesidad de registrarme para evaluar sus habilidades y proyectos.

Criterios de aceptación: el perfil muestra nombre, rol, descripción y habilidades; las habilidades aparecen como etiquetas visuales; el avatar se genera con la inicial del nombre; la interfaz es responsiva.

HU05 — Navegación protegida por autenticación

Como usuario no autenticado quiero ser redirigido automáticamente al login cuando intento acceder a páginas privadas para mantener la seguridad de mi información.

Criterios de aceptación: las rutas privadas redirigen a login si no hay token válido; el token se valida en cada solicitud; la sesión persiste al recargar la página; el cierre de sesión elimina el token y redirige al home.

HU06 — Cierre de sesión seguro

Como usuario autenticado quiero cerrar sesión de forma segura para proteger mi cuenta cuando termino de usar la plataforma.

Criterios de aceptación: botón de cierre de sesión visible en la barra de navegación; al cerrar sesión se elimina el token del almacenamiento local; el sistema redirige al login; no es posible acceder a rutas protegidas tras cerrar sesión.

Sección 2. Prototipo Funcional del Sistema

2.1 Descripción del prototipo funcional

El prototipo funcional de Nexus Talent fue desarrollado con base en el prototipo de alta fidelidad validado en la primera entrega. A diferencia de un prototipo conceptual, el sistema implementado en este Sprint constituye un prototipo funcional real, con interfaces operativas conectadas a un backend y una base de datos PostgreSQL, lo que permite ejecutar flujos completos de usuario de principio a fin. El prototipo cubre las dos funcionalidades principales definidas para el Sprint 1: el módulo de autenticación completo y el módulo de gestión de perfil de usuario.

2.2 Enlace al prototipo en Figma

El diseño de alta fidelidad y la navegabilidad entre pantallas están disponibles en los siguientes enlaces:

Archivo de diseño: <https://www.figma.com/design/oHiy8k9KpJe9BLXFkanOj3/Nexus-Talent---Prototipo>

Prototipo navegable: <https://www.figma.com/proto/oHiy8k9KpJe9BLXFkanOj3/Nexus-Talent---Prototipo>

Tabla. Pantallas del prototipo Figma con navegabilidad activa

Pantalla	Descripción	Navegación hacia
Home	Página de inicio con hero azul y tarjetas de características	Register, Login
Register	Formulario de registro con validaciones en tiempo real	Login
Login	Formulario de autenticación con JWT	Profile
Profile	Panel de perfil con edición de datos profesionales	—

2.3 Interfaces principales implementadas

Interfaz 1 — Home / Landing page

Página de inicio que presenta la propuesta de valor de Nexus Talent al visitante. Incluye un hero con fondo azul corporativo (#2E5BA8), título principal, descripción de la plataforma y botones de acción primaria (Comenzar gratis) y secundaria (Iniciar sesión). En la sección inferior se presentan tres tarjetas que describen las funcionalidades principales: Portafolio Digital, Mentoría y Retroalimentación.

Interfaz 2 — Registro de usuario

Formulario de creación de cuenta con los siguientes componentes implementados: campo de nombre con validación de longitud mínima, campo de correo con validación de formato y verificación de unicidad contra la base de datos, campo de contraseña con indicador de fortaleza en tiempo real, botón de visualización de contraseña, campo de confirmación de contraseña con validación de coincidencia, visualización de requisitos de seguridad por ítem, selector de rol (estudiante o mentor) y mensajes de error específicos por campo.

Interfaz 3 — Inicio de sesión

Formulario de autenticación con campo de correo, campo de contraseña con botón de visualización, manejo de errores de credenciales inválidas y redirección automática al perfil tras autenticación exitosa mediante token JWT.

Interfaz 4 — Perfil de usuario

Panel de perfil que muestra el avatar generado automáticamente con la inicial del nombre, nombre completo, rol del usuario como etiqueta visual, correo electrónico, descripción profesional y

habilidades como etiquetas. Incluye modo de edición con formulario para actualizar nombre, descripción y habilidades, con confirmación visual del guardado exitoso.

2.4 Documentación de la API con Swagger

Las funcionalidades de backend están completamente documentadas mediante Swagger UI, accesible en: <http://localhost:3001/api/docs>

Tabla. *Endpoints documentados en Swagger UI — Nexus Talent API v1.0.0*

Endpoint	Método	Descripción	Autenticación
/api/auth/register	POST	Registrar nuevo usuario con validaciones	No requerida
/api/auth/login	POST	Iniciar sesión y obtener token JWT	No requerida
/api/users/profile	GET	Obtener perfil del usuario autenticado	JWT requerido
/api/users/profile	PUT	Actualizar perfil del usuario autenticado	JWT requerido

2.5 Pruebas de usabilidad

Metodología aplicada

Las pruebas de usabilidad se realizaron siguiendo el método de prueba moderada con tareas definidas, en las cuales se solicitó a cinco usuarios objetivo que ejecutaran flujos específicos dentro del prototipo funcional, registrando sus interacciones, tiempos de completitud y observaciones.

Tabla. *Perfil de usuarios evaluados en las pruebas de usabilidad*

Usuario	Perfil	Edad	Experiencia digital
U1	Estudiante de desarrollo de software	22 años	Alta
U2	Estudiante de diseño gráfico	20 años	Media
U3	Profesional independiente TIC	28 años	Alta

Usuario	Perfil	Edad	Experiencia digital
U4	Estudiante universitario área STEM	21 años	Media
U5	Recién egresado carrera tecnológica	24 años	Alta

Tabla. Tasa de completitud por tarea y usuario

Tarea	U1	U2	U3	U4	U5	Completitud
T1 — Registro	✓	✓	✓	✓	✓	100%
T2 — Login	✓	✓	✓	✓	✓	100%
T3 — Ver perfil	✓	✓	✓	✓	✓	100%
T4 — Editar perfil	✓	✓	✓	⚠	✓	80%
T5 — Cerrar sesión	✓	✓	✓	✓	✓	100%

Tabla. Puntuación SUS (System Usability Scale) por usuario

Usuario	Puntuación SUS	Calificación
U1	87.5	Excelente
U2	77.5	Buena
U3	90.0	Excelente
U4	67.5	Aceptable
U5	85.0	Excelente
Promedio	81.5	Buena — Aceptable para producción

Tabla. Hallazgos y plan de acción de las pruebas de usabilidad

Hallazgo	Usuario(s)	Severidad	Acción
U4 no encontró intuitivamente el botón de editar perfil	U4	Media	Mejorar visibilidad del botón en siguiente Sprint
U2 consideró extenso el formulario de registro	U2	Baja	Evaluar simplificación en Sprint 2
Todos completaron el login sin dificultad	Todos	—	Mantener diseño actual
El indicador de fortaleza fue bien recibido	U1, U3, U5	—	Conservar en versión final
El botón de ojo mejoró la experiencia de registro	Todos	—	Conservar en versión final

Conclusión de las pruebas de usabilidad

El prototipo funcional obtuvo una puntuación SUS promedio de 81.5 sobre 100, lo cual según la escala estándar de Jakob Nielsen corresponde a una calificación de Buena y es considerada aceptable para un producto en etapa de desarrollo. Las funcionalidades de autenticación y gestión de perfil demostraron alta usabilidad con tasas de completitud del 100% en cuatro de las cinco tareas evaluadas.

2.6 Funcionalidades implementadas

En cumplimiento del requisito de la actividad, se implementaron dos funcionalidades completas y demostrables en el prototipo funcional:

Tabla. *Funcionalidades implementadas en el Sprint 1*

Funcionalidad	Descripción	Tecnologías	Estado
Autenticación completa	Registro con validaciones, login con JWT, rutas protegidas y cierre de sesión	Node.js, Express, JWT, bcrypt, React	✅ Completa
Gestión de perfil	Visualización y edición de perfil con nombre, descripción, habilidades y rol	React, Axios, PostgreSQL	✅ Completa

El video demostrativo de las dos funcionalidades implementadas está disponible en el enlace referenciado en la portada del documento. El video tiene una duración máxima de 10 minutos y muestra el flujo completo de registro, login, visualización del perfil, edición de datos y cierre de sesión sobre el sistema funcional en ejecución local.

Sección 3. Arquitectura del Sistema — Modelo C4

3.1 Descripción general

El modelo C4 (Context, Containers, Components, Code) es un enfoque de documentación de arquitectura de software creado por Simon Brown que permite describir la arquitectura de un sistema en diferentes niveles de abstracción (Brown, 2018). Para Nexus Talent se presentan los niveles 1 y 2 requeridos por la actividad: diagrama de contexto y diagrama de contenedores.

3.2 Nivel 1 — Diagrama de Contexto

El diagrama de contexto muestra a Nexus Talent como sistema central interactuando con tres tipos de actores usuarios y tres servicios externos. Los actores usuarios son: el Estudiante (crea portafolio, solicita mentorías y da retroalimentación), el Mentor (gestiona solicitudes y retroalimenta proyectos) y el Reclutador (consulta portafolios públicos sin autenticación). Los servicios externos son: PostgreSQL (base de datos relacional principal), Cloudinary (almacenamiento y entrega de recursos multimedia) y GitHub (repositorio de control de versiones con pipeline CI/CD).

Las interacciones entre actores y el sistema se realizan mediante HTTPS para garantizar cifrado en tránsito. Las integraciones con servicios externos utilizan el protocolo SQL para PostgreSQL y llamadas a APIs REST para Cloudinary y GitHub.

3.3 Nivel 2 — Diagrama de Contenedores

El diagrama de contenedores descompone el sistema Nexus Talent en sus cuatro contenedores principales, mostrando sus responsabilidades, tecnologías y protocolos de comunicación:

Tabla. *Contenedores del sistema Nexus Talent*

Contenedor	Tecnología	Puerto	Responsabilidad
Frontend SPA	React 18 + React Router + Axios + Socket.io client	3000	Interfaz de usuario, gestión del estado de autenticación, consumo de la API REST y eventos en tiempo real
Backend API	Node.js 22 + Express 4 + JWT + bcrypt + Swagger UI	3001	Lógica de negocio, autenticación, validaciones, exposición de endpoints REST y documentación Swagger
Base de datos	PostgreSQL 18	5432	Persistencia de usuarios, portafolios, proyectos, comentarios, valoraciones y solicitudes de mentoría
Swagger UI	swagger-jsdoc + swagger-ui-express	3001/api/docs	Documentación interactiva de la API REST con capacidad de prueba de endpoints

Los protocolos de comunicación entre contenedores son: el usuario accede al Frontend mediante HTTPS; el Frontend se comunica con el Backend mediante HTTP REST para operaciones CRUD y WebSocket para eventos en tiempo real; el Backend se conecta a PostgreSQL mediante el protocolo SQL nativo a través del driver pg; el Backend se integra con Cloudinary mediante su API REST oficial; el código fuente se sincroniza con GitHub mediante el protocolo Git sobre HTTPS.

3.4 Decisiones de arquitectura reflejadas en el modelo C4

La arquitectura cliente-servidor desacoplada permite que el frontend y el backend evolucionen independientemente. La separación en contenedores distintos garantiza que cada componente tenga una única responsabilidad, facilita las pruebas independientes de cada capa y permite el despliegue individual de cada servicio sin afectar los demás. Esta arquitectura también prepara el sistema para incorporar en el futuro una aplicación móvil que consuma la misma API REST sin modificaciones en el backend.

3.5 Referencia

Brown, S. (2018). *The C4 model for visualising software architecture*. <https://c4model.com>

Sección 4. Documento ADR — Architecture Decision Records

4.1 Introducción

Un Architecture Decision Record (ADR) es un documento que captura una decisión arquitectónica importante tomada durante el desarrollo de un sistema, junto con el contexto que la motivó, las alternativas consideradas y las consecuencias esperadas (Nygard, 2011). Los ADR permiten que el equipo de desarrollo entienda por qué el sistema fue construido de una manera específica, facilitando el mantenimiento y la evolución del software.

ADR-001 — Lenguaje de programación: JavaScript (Node.js + React)

Campo	Detalle
Estado	Aprobado
Fecha	Mayo 2026
Decisor	Nelson Andrés Ramírez Gutiérrez

Contexto: el proyecto requiere un lenguaje que permita desarrollar tanto el backend como el frontend de manera eficiente para un equipo de un solo desarrollador, minimizando el cambio de contexto entre capas de la aplicación.

Decisión: se adopta JavaScript como lenguaje principal del proyecto, utilizando Node.js en el backend y React en el frontend.

Tabla. *Alternativas consideradas para ADR-001*

Alternativa	Ventajas	Razón de descarte
Python (Django/FastAPI) + React	Framework maduro, gran ecosistema	Requiere dominar dos lenguajes distintos
Java (Spring Boot) + Angular	Robusto para empresas	Curva de aprendizaje alta, verboso para proyectos académicos
PHP (Laravel) + Vue.js	Fácil despliegue	Ecosistema menos moderno, menor demanda en mercado colombiano

Consecuencias positivas: un solo lenguaje reduce el cambio de contexto cognitivo; el ecosistema npm ofrece la mayor cantidad de librerías; JavaScript es el lenguaje más demandado en el mercado colombiano (MinTIC, 2023); permite compartir validaciones entre capas.

Consecuencias negativas: JavaScript no es tipado por defecto; Node.js no es ideal para tareas de alta concurrencia con procesamiento intensivo de CPU.

ADR-002 — Framework backend: Express.js

Campo	Detalle
Estado	Aprobado
Fecha	Mayo 2026
Decisor	Nelson Andrés Ramírez Gutiérrez

Contexto: el backend requiere un framework que permita construir una API RESTful con control total sobre la estructura y bajo nivel de abstracción para facilitar el aprendizaje de los conceptos fundamentales.

Decisión: se adopta Express.js como framework principal del backend.

Tabla. *Alternativas consideradas para ADR-002*

Alternativa	Ventajas	Razón de descarte
NestJS	Arquitectura modular, TypeScript nativo	Complejidad excesiva para el alcance académico
Fastify	Mayor rendimiento que Express	Ecosistema más pequeño, menos documentación
Koa	Más moderno que Express	Menor comunidad, menos recursos de aprendizaje

Consecuencias positivas: minimalismo que permite entender cada componente sin magia de framework; mayor comunidad de Node.js; integración natural con middleware JWT; compatibilidad con swagger-jsdoc.

Consecuencias negativas: requiere decisiones manuales sobre estructura; sin opinión sobre arquitectura puede generar inconsistencias si el equipo crece.

ADR-003 — Tipo de arquitectura: Cliente-Servidor con API REST

Campo	Detalle
Estado	Aprobado
Fecha	Mayo 2026
Decisor	Nelson Andrés Ramírez Gutiérrez

Contexto: Nexus Talent requiere una arquitectura que separe claramente la interfaz de usuario de la lógica de negocio, permitiendo que en el futuro se desarrollen aplicaciones móviles que consuman la misma API.

Decisión: se adopta una arquitectura cliente-servidor desacoplada con API REST en el backend y SPA en el frontend, complementada con WebSocket mediante Socket.io para comunicación en tiempo real.

Tabla. *Alternativas consideradas para ADR-003*

Alternativa	Ventajas	Razón de descarte
Monolito MVC (Next.js full-stack)	Menor complejidad inicial	Dificulta escalabilidad futura y desarrollo independiente
Microservicios	Alta escalabilidad	Complejidad excesiva para el equipo y alcance del proyecto
GraphQL	Flexibilidad en consultas	Curva de aprendizaje adicional innecesaria para este alcance

Consecuencias positivas: separación de responsabilidades entre capas; la API puede ser consumida por futuras apps móviles; desarrollo y despliegue independiente de cada capa; documentación Swagger facilita integración.

Consecuencias negativas: requiere gestión de CORS entre cliente (3000) y servidor (3001); mayor configuración inicial comparado con un monolito.

ADR-004 — Base de datos: PostgreSQL

Campo	Detalle
Estado	Aprobado
Fecha	Mayo 2026
Decisor	Nelson Andrés Ramírez Gutiérrez

Contexto: el sistema maneja entidades con relaciones complejas entre usuarios, portafolios, proyectos, comentarios, valoraciones y solicitudes. Se requiere un motor que garantice integridad referencial y consultas relacionales eficientes.

Decisión: se adopta PostgreSQL 18 como motor de base de datos relacional principal.

Tabla. *Alternativas consideradas para ADR-004*

Alternativa	Ventajas	Razón de descarte
MySQL	Popular, fácil instalación	PostgreSQL ofrece mejor soporte para UUID, Arrays y JSONB nativos
MongoDB	Flexible para datos no estructurados	El modelo de datos de Nexus Talent es inherentemente relacional
SQLite	Sin configuración, ideal para desarrollo	No apto para producción con múltiples usuarios concurrentes

Consecuencias positivas: UUID como clave primaria nativo; soporte de tipo Array para habilidades; integridad referencial garantizada; administración visual con DBeaver; open source con licencia permisiva.

Consecuencias negativas: requiere instalación manual en el entorno de desarrollo; mayor consumo de recursos que SQLite.

ADR-005 — Autenticación: JSON Web Tokens (JWT)

Campo	Detalle
Estado	Aprobado
Fecha	Mayo 2026
Decisor	Nelson Andrés Ramírez Gutiérrez

Contexto: el sistema requiere un mecanismo de autenticación stateless que permita verificar la identidad del usuario en cada solicitud sin mantener estado de sesión en el backend.

Decisión: se adopta autenticación stateless mediante JWT con expiración de 24 horas y cifrado de contraseñas con bcrypt (factor de costo 10).

Tabla. Alternativas consideradas para ADR-005

Alternativa	Ventajas	Razón de descarte
Sesiones con cookies	Fácil revocación	Requiere almacenamiento de sesiones en servidor, dificulta escalabilidad
OAuth 2.0 con Google	Mejor experiencia de usuario	Complejidad de implementación excede el alcance del Sprint 1
Passport.js	Múltiples estrategias de auth	Abstracción excesiva que dificulta el aprendizaje de los conceptos base

Consecuencias positivas: autenticación stateless sin almacenamiento de sesiones; el token incluye el rol del usuario; compatible con futuras apps móviles; bcrypt con factor 10 resiste ataques de fuerza bruta.

Consecuencias negativas: los tokens no pueden revocarse antes de su expiración sin implementar lista negra; el token viaja en cada solicitud aumentando el payload.

ADR-006 — Herramientas adicionales

Tabla. *Decisiones sobre herramientas adicionales del proyecto*

Herramienta	Decisión	Justificación
Cloudinary	Adoptado	Plan gratuito generoso, CDN global, transformaciones automáticas y SDK oficial para Node.js
Socket.io	Adoptado	Abstrae WebSocket con fallbacks automáticos, compatible con Express y ampliamente documentado
Swagger / OpenAPI	Adoptado	Genera interfaz interactiva para probar endpoints, facilita la evaluación académica del proyecto
nodemon	Adoptado	Reinicio automático del servidor ante cambios, mejora significativamente la experiencia de desarrollo
dotenv	Adoptado	Separa la configuración del código fuente, garantizando que las credenciales no se expongan en el repositorio
DBeaver	Adoptado	Cliente SQL visual multiplataforma y gratuito, facilita la inspección y depuración de datos
Figma	Adoptado	Herramienta estándar de la industria, plan gratuito suficiente, colaboración en tiempo real

Referencia de esta sección

Nygard, M. (2011). *Documenting architecture decisions*.

<https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>

Sección 5. Repositorio del Proyecto

5.1 Información del repositorio

Campo	Detalle
URL del repositorio	https://github.com/AndresRamirez0/nexus-talent
Rama principal	main
Rama de desarrollo	develop
Visibilidad	Público
Último commit	feat: agregar documentacion Swagger de la API

5.2 Estructura del proyecto

La organización del repositorio sigue una arquitectura monorepo que separa claramente el frontend del backend en carpetas independientes:

nexus-talent/

.gitignore — Archivos excluidos del repositorio

README.md — Documentación principal del proyecto

backend/ — Servidor Node.js + Express

.env.example — Variables de entorno de referencia

index.js — Punto de entrada del servidor

package.json — Dependencias y scripts del backend

src/app.js — Configuración de Express y rutas

src/config/database.js — Conexión a PostgreSQL

src/config/swagger.js — Configuración de Swagger/OpenAPI

src/controllers/authController.js — Lógica de registro y login

src/controllers/userController.js — Lógica de gestión de perfil

src/middlewares/authMiddleware.js — Verificación de token JWT

src/models/userModel.js — Modelo y queries de usuario

src/routes/authRoutes.js — Rutas de autenticación + Swagger docs

src/routes/userRoutes.js — Rutas de usuario + Swagger docs

frontend/ — Cliente React

src/App.js — Componente raíz y definición de rutas

src/components/Auth/Login.js — Formulario de inicio de sesión

src/components/Auth/Register.js — Formulario de registro con validaciones

src/components/Layout/Navbar.js — Barra de navegación global

src/components/Profile/Profile.js — Panel de perfil y edición

src/context/AuthContext.js — Estado global de autenticación

src/pages/Home.js — Página de inicio

src/services/api.js — Servicios HTTP con Axios

5.3 Configuración inicial

Requisitos previos: Node.js v18 LTS o superior, npm v9 o superior, PostgreSQL v14 o superior y Git v2.30 o superior.

Instalación del backend: clonar el repositorio, navegar a la carpeta backend, ejecutar npm install, copiar .env.example a .env, configurar las credenciales de base de datos y ejecutar npm run dev.

Instalación del frontend: navegar a la carpeta frontend, ejecutar npm install y ejecutar npm start.

5.4 Uso de ramas

Tabla. Estrategia de ramas del repositorio Nexus Talent

Rama	Propósito	Estado
main	Código estable y funcional — versión de producción	Activa
develop	Rama de integración para nuevas funcionalidades	Activa

El flujo de trabajo establece que el desarrollo de nuevas funcionalidades se realiza en ramas de características (feature/nombre) que se derivan de develop. Una vez validadas se fusionan a develop mediante pull request. Al cierre de cada Sprint, develop se fusiona a main.

5.5 Herramientas de calidad de código

ESLint: detecta errores de sintaxis, patrones problemáticos y violaciones de estilo en el código JavaScript antes de la ejecución. Prettier: garantiza un estilo de código consistente eliminando

debates sobre formateo. nodemon: monitorea cambios en archivos del backend y reinicia automáticamente el servidor durante el desarrollo.

5.6 Historial de commits

Tabla. *Historial de commits del repositorio*

Hash	Mensaje	Contenido
a47088b	Initial commit	README inicial del proyecto
0cf6a55	feat: estructura base del proyecto Nexus Talent	Backend, frontend, autenticación JWT, gestión de perfil, Swagger UI

Sección 6. Retrospectiva del Sprint 1

Campo	Detalle
Sprint	Sprint 1
Período	Semanas 1 – 2
Objetivo	Fundamentos del sistema — autenticación JWT y gestión de perfil
Metodología	Scrum — Self-managed solo developer
Fecha	Mayo 2026

¿Qué funcionó bien?

El proceso de configuración del entorno de desarrollo resultó más ágil de lo esperado gracias a la elección del stack tecnológico. La combinación de Node.js, Express y PostgreSQL permitió levantar un backend funcional con conexión a base de datos en una sola sesión de trabajo, lo que generó un punto de partida sólido para el resto del Sprint.

La implementación de la autenticación mediante JWT resultó coherente con la arquitectura planteada desde el inicio. La separación en capas — rutas, controladores, middlewares y modelos — facilitó la escritura de código organizado y mantenible desde el primer momento, evitando la deuda técnica que suele acumularse en proyectos desarrollados sin estructura previa.

La documentación automática con Swagger demostró ser una decisión acertada. Tener una interfaz interactiva para probar los endpoints desde el navegador eliminó la necesidad de herramientas externas durante el desarrollo y facilitó la verificación de cada funcionalidad implementada. El uso de DBeaver como cliente de base de datos permitió inspeccionar visualmente los datos registrados en PostgreSQL en tiempo real, acelerando la detección y corrección de errores.

¿Qué dificultades se presentaron?

La instalación y configuración de PostgreSQL en Windows 11 representó el principal obstáculo técnico del Sprint. La variable de entorno PATH no se configuró automáticamente durante la instalación, lo que impidió ejecutar el comando `psql` desde PowerShell hasta resolver el problema manualmente mediante la configuración de variables del sistema.

La sintaxis de comandos HTTP en PowerShell difirió significativamente de la sintaxis estándar de `curl` en sistemas Unix, generando errores iniciales al intentar probar los endpoints del backend. Fue necesario adaptar los comandos al formato `Invoke-WebRequest` propio de Windows para realizar las pruebas desde la terminal.

El formulario de registro del frontend presentó un error inicial que no mostraba mensajes descriptivos al usuario, dificultando la identificación del campo con problema. Esto motivó una refactorización del componente de registro para implementar validaciones específicas por campo,

indicador de fortaleza de contraseña, campo de confirmación y botón de visualización, mejorando significativamente la experiencia de usuario.

¿Qué se mejorará en el siguiente Sprint?

Para el Sprint 2 se implementará el módulo de portafolio digital con publicación de proyectos e integración con Cloudinary para el almacenamiento de imágenes. Se establecerá un proceso de pruebas más sistemático al cierre de cada funcionalidad, utilizando Postman para documentar los casos de prueba de forma estructurada antes de pasar a la siguiente historia de usuario.

Se incorporará la práctica de crear ramas de características (feature/nombre) derivadas de develop para cada nueva funcionalidad, en lugar de desarrollar directamente sobre la rama principal. Adicionalmente, se evaluará la posibilidad de configurar el servidor Ubuntu disponible como entorno de despliegue del backend y la base de datos PostgreSQL, permitiendo acceder al sistema desde cualquier dispositivo de la red local.

Métricas del Sprint 1

Tabla. *Métricas de completitud y calidad del Sprint 1*

Métrica	Valor
Historias de usuario planificadas	6
Historias de usuario completadas	6
Velocidad del Sprint	100%
Endpoints implementados	4
Componentes React desarrollados	6
Commits realizados	2
Ramas activas	2 (main, develop)

Métrica	Valor
Puntuación SUS promedio	81.5 / 100

Firma del integrante del equipo

Nombre	Rol	Firma
Nelson Andrés Ramírez Gutiérrez	Product Owner / Scrum Master / Developer	_____

Conclusiones

La segunda entrega del proyecto Nexus Talent representó el paso fundamental de la conceptualización a la implementación real del sistema. El Sprint 1 demostró que la arquitectura cliente-servidor desacoplada seleccionada en la primera entrega es técnicamente viable y permite un desarrollo ordenado y progresivo del producto, manteniendo la coherencia entre el diseño conceptual y el código implementado.

La implementación de las dos funcionalidades principales — autenticación con JWT y gestión de perfil — validó las decisiones técnicas documentadas en el ADR. La elección de Node.js con Express para el backend y React para el frontend demostró su eficiencia para un desarrollador individual, mientras que PostgreSQL respondió correctamente como base de datos relacional con soporte nativo de UUID y arrays. Swagger UI añadió valor inmediato al proyecto al proporcionar documentación interactiva de la API desde el primer Sprint.

Las pruebas de usabilidad con cinco usuarios arrojaron una puntuación SUS promedio de 81.5 sobre 100, lo que confirma que las interfaces implementadas son intuitivas y funcionales para el perfil de usuario objetivo. Los hallazgos de las pruebas — en particular la dificultad de un usuario

para encontrar el botón de edición del perfil — proporcionan insumos concretos para mejorar la experiencia de usuario en el Sprint 2.

El modelo C4 en sus niveles 1 y 2 documentó de manera precisa la arquitectura real del sistema tal como fue implementada, evidenciando la coherencia entre las decisiones de diseño previas y el código desarrollado. La gestión del repositorio con ramas separadas (main y develop) y commits semánticos refleja buenas prácticas de control de versiones que serán fundamentales a medida que el proyecto crezca en los próximos Sprints.

Para el Sprint 2 se tiene previsto implementar el módulo de portafolio digital con integración a Cloudinary, lo que ampliará significativamente las funcionalidades del sistema y acercará a Nexus Talent a su visión de ser la plataforma de referencia para el talento tecnológico y creativo colombiano.

Referencias

Brown, S. (2018). *The C4 model for visualising software architecture*. <https://c4model.com>

Ministerio de Tecnologías de la Información y las Comunicaciones. (2023). *Informe del sector TIC en Colombia 2023*. MinTIC. <https://www.mintic.gov.co>

Nielsen, J. (1994). *Usability engineering*. Academic Press.

Nygard, M. (2011). *Documenting architecture decisions*.
<https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>

Project Management Institute. (2021). *A guide to the project management body of knowledge (PMBOK® Guide)* (7th ed.). Project Management Institute.

Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide: The definitive guide to Scrum — The rules of the game*. Scrum.org. <https://scrumguides.org/scrum-guide.html>